Lab 4: Candidate moves in local search

Authors: Piotr Foltyniewicz 160295, Jakub Adamski 160291

Problem description

Each instance consists of 200 nodes. A row in the input file contains the node's planar coordinates and selection cost, in the form `x;y;cost`. Node indices are zero-based and correspond to row order. The task is to select exactly 100 distinct nodes (half of the available nodes), order them, and form a closed cycle that minimizes the sum of travel distances and selected-node costs.

Distances are Euclidean, rounded to the nearest integer and precomputed after reading the instance. All algorithms use this distance matrix. The evaluation sums all selected-node costs and all cycle edges, including last -> first. In the pseudocode, `m = 100`, `n = 200`, `c[v]` is node cost, and `d(a, b)` is the precomputed distance. A solution `S` is an ordered list of `m` distinct nodes; `U` denotes the set of nodes not in `S`.

The objective for an ordered selection `S` is:

```
F(S) = sum(c[S[i]]) + sum(d(S[i], S[(i + 1) mod length(S)]))
```

Pseudocode

All indices are zero-based. We use steepest search with inter-route replacement and 2-opt. The baseline checks all moves; candidate search checks only moves that add a candidate edge. Moves are evaluated through deltas without constructing neighboring solutions.

Candidate edges

For each node `u`, choose `k` other nodes minimizing `d(u, v) + c[v]`, breaking ties by node ID. `A[u]` stores candidate neighbors: `v` belongs to it if `u` chose `v` or `v` chose `u`. These lists are built once and remain unchanged during the search. Adding reverse connections can make `A[u]` longer than `k`.

```
BUILD_CANDIDATES(k):
    A = empty candidate lists
    for u in all nodes:
        C[u] = k smallest v != u by (d(u, v) + c[v], v)
        for v in C[u]:
            add v to A[u] and u to A[v], without duplicates
    return A
```

We use k = 10 and compare it with k = 5 and k = 20.

Neighborhood and deltas

`S` is the ordered cycle and `A[node]` lists candidates for `node`. `pos[node]` is its position in `S`; `-1` means the node is outside `S`. Indices `i`, `j`, `p`, `q` are cycle positions, and `prev` / `next` refer to their cyclic neighbors.

- `INTER(p, v)`: replace `S[p]` with a node `v` from outside `S`.
- `TWO_OPT(i, j)`: order the cut positions so `i < j`, remove the edges starting there, reconnect the cycle, and reverse `S[i+1..j]`. The cut edges must not share a node.

If `candidate` is outside `S`, replacing either neighbor of `node` creates edge `{node, candidate}`. If both nodes are in `S`, consider the two 2-opt moves: cutting after both nodes or before both nodes. The condition `node < candidate` processes the pair only once, rather than again in the opposite order.

```
CANDIDATE_MOVES(S, A):
    moves = []
    for node from 0 to n - 1:
        pos[node] = -1
    for i from 0 to m - 1:
        pos[S[i]] = i

    for i from 0 to m - 1:
        node = S[i]
        for candidate in A[node]:
            if pos[candidate] == -1:
                add INTER(prev(i), candidate) to moves
                add INTER(next(i), candidate) to moves
            else if node < candidate:
                j = pos[candidate]
                if edges (S[i], S[next(i)]) and (S[j], S[next(j)]) do not share a node:
                    add TWO_OPT(i, j) to moves

                p = prev(i); q = prev(j)
                if edges (S[p], S[next(p)]) and (S[q], S[next(q)]) do not share a node:
                    add TWO_OPT(p, q) to moves
    return moves

DELTA_INTER(S, p, v):
    out = S[p]; a = S[prev(p)]; b = S[next(p)]
    return d(a,v) + d(v,b) - d(a,out) - d(out,b) + c[v] - c[out]

DELTA_TWO_OPT(S, i, j):
    a = S[i]; b = S[next(i)]; e = S[j]; f = S[next(j)]
    return d(a,e) + d(b,f) - d(a,b) - d(e,f)
```

Steepest local search

`DELTA` uses the formula for the move type. `APPLY` performs the replacement or reversal. Equal improving deltas prefer inter-route moves, then smaller position and node indices.

```
CANDIDATE_STEEPEST(S, A):
    value = F(S)
    loop:
        best = NONE; bestDelta = 0
        for move in CANDIDATE_MOVES(S, A):
            delta = DELTA(S, move)
            if delta < bestDelta: best = move; bestDelta = delta
        if best == NONE: return S, value
        APPLY(S, best)
        value += bestDelta
```

Starting solutions

Random starts contain the first 100 nodes of a shuffled permutation of all 200 nodes. Each method uses the same 200 starting cycles per instance, generated with seed 20261031. Reported times include creating the start and performing local search; candidate lists are built before the measured runs.

Results

Each of the four methods was run 200 times on each instance. Results of labs 1-3 are copied from their current summaries and marked with the lab number; methods measured in lab 4 are in bold. The lab 4 baseline and the historical lab 3 result are shown separately because they use different samples of random starts.

Objective values use the format **average (minimum – maximum)**; running times are averages per run in milliseconds. Lower objective values are better. The lowest minimum within each lab and instance is in bold.

Objective function values

Method	TSPA	TSPB
Steepest Edges Random (baseline)	76126.85 (**73288** – 79682)	52712.36 (49816 – 55429)
Candidate Edges k5 Random	76993.97 (74191 – 80983)	52818.47 (50722 – 56889)
Candidate Edges k10 Random	76278.36 (73966 – 79184)	52640.37 (50362 – 56836)
Candidate Edges k20 Random	76095.42 (**73288** – 80182)	52675.29 (**49459** – 55512)
Steepest Nodes Random (lab 3)	90017.03 (82067 – 97807)	65860.14 (58859 – 72495)
Steepest Nodes Greedy (lab 3)	73021.71 (**71955** – 74861)	54292.07 (49978 – 61265)
Steepest Edges Random (lab 3)	76234.49 (73409 – 79907)	52513.55 (50126 – 55265)
Steepest Edges Greedy (lab 3)	73003.80 (**71955** – 74861)	53091.28 (**49346** – 60354)
Greedy Nodes Random (lab 3)	88281.52 (83276 – 97988)	63685.80 (57875 – 70381)
Greedy Nodes Greedy (lab 3)	73021.00 (**71955** – 74861)	54258.15 (49978 – 61265)
Greedy Edges Random (lab 3)	76362.62 (74427 – 78668)	52948.87 (50667 – 56093)
Greedy Edges Greedy (lab 3)	72983.31 (**71955** – 74877)	53151.86 (49546 – 60357)
Greedy Cycle 2-regret heuristics (lab 2)	120378.77 (112806 – 128089)	72484.84 (68150 – 77274)
Greedy Cycle heuristics with a weighted sum criterion (lab 2)	73353.51 (**72011** – 75532)	57866.33 (52731 – 63370)

NN heuristics 2-regret heuristics (lab 2)	121360.29 (110950 – 135771)	73649.84 (68761 – 79756)
NN heuristics with a weighted sum criterion (lab 2)	75561.20 (72419 – 78504)	55698.34 (**50485** – 61406)
Random (lab 1)	262009.64 (234223 – 288919)	206071.06 (181689 – 230681)
Nearest Node End (lab 1)	87476.63 (84362 – 90075)	59397.34 (56007 – 63166)
Nearest Node Any (lab 1)	74306.53 (**72328** – 77022)	59476.23 (**52915** – 66164)
Greedy Cycle (lab 1)	73743.73 (72590 – 75281)	59650.81 (55655 – 63734)

Running times (ms)

Total times cover the complete run: creating the starting solution and performing local search. Candidate lists are built once per instance and method, outside the measured runs. Starting-solution times for lab 3 are copied from its current report. Empty cells mean that starting-solution time was not measured separately (lab 4) or that there is no separate starting-solution stage (labs 1-2).

Labs 1-3 retain their original measurements. Lab 4 was measured separately, so speedups are calculated against the baseline rerun in lab 4, using the same starts and environment as the candidate methods.

Method	TSPA: total	TSPA: starting solution	TSPB: total	TSPB: starting solution
Steepest Edges Random (baseline)	33.799286		34.286283	
Candidate Edges k5 Random	4.173085		3.818066	
Candidate Edges k10 Random	7.909238		6.727505	
Candidate Edges k20 Random	14.563482		12.108440	
Steepest Nodes Random (lab 3)	66.795616	0.003531	66.738804	0.003519
Steepest Nodes Greedy (lab 3)	5.276252	4.066111	8.437412	4.111100
Steepest Edges Random (lab 3)	34.488630	0.003531	35.122619	0.003519
Steepest Edges Greedy (lab 3)	4.453417	4.066111	7.405829	4.111100
Greedy Nodes Random (lab 3)	27.378438	0.003531	29.323703	0.003519
Greedy Nodes Greedy (lab 3)	5.271248	4.066111	7.354972	4.111100
Greedy Edges Random (lab 3)	24.915344	0.003531	25.450824	0.003519
Greedy Edges Greedy (lab 3)	4.911828	4.066111	7.588654	4.111100
Greedy Cycle 2-regret heuristics (lab 2)	3.767999		3.729813	

Greedy Cycle heuristics with a weighted sum criterion (lab 2)	3.722352		3.621625	
NN heuristics 2-regret heuristics (lab 2)	3.826779		3.824544	
NN heuristics with a weighted sum criterion (lab 2)	3.893400		4.053285	
Random (lab 1)	0.032951		0.013712	
Nearest Node End (lab 1)	0.256396		0.118587	
Nearest Node Any (lab 1)	3.829626		3.895479	
Greedy Cycle (lab 1)	2.859520		3.091238	

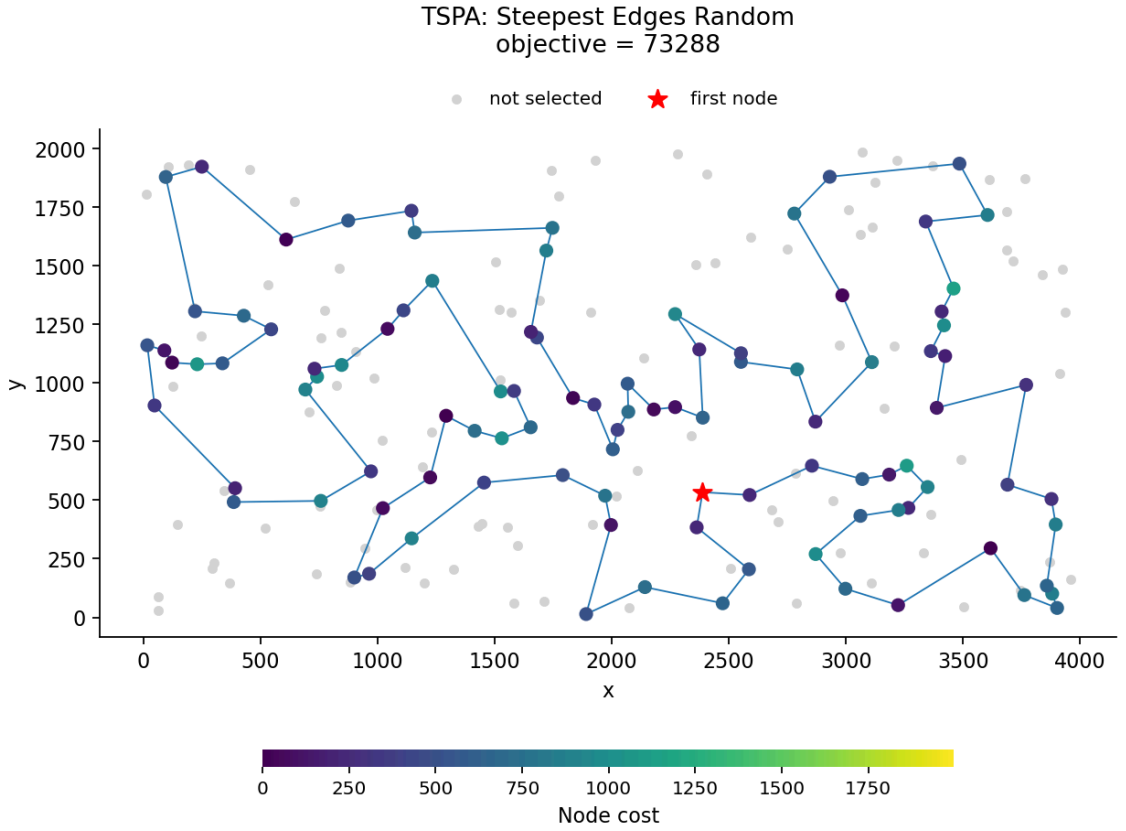
Best solutions and visualizations

Node color represents selection cost; the compact horizontal scale below each plot uses the same cost range for all methods on an instance. Gray nodes are not selected, blue segments form the closed tour, and the red star marks the first node in the displayed sequence. Lists below show ten node indices per row; read left to right, then top to bottom to follow the tour. Only methods measured in lab 4 are visualized; historical results from labs 1-3 appear only in the Results tables.

TSPA

Steepest Edges Random

Objective: **73288**.

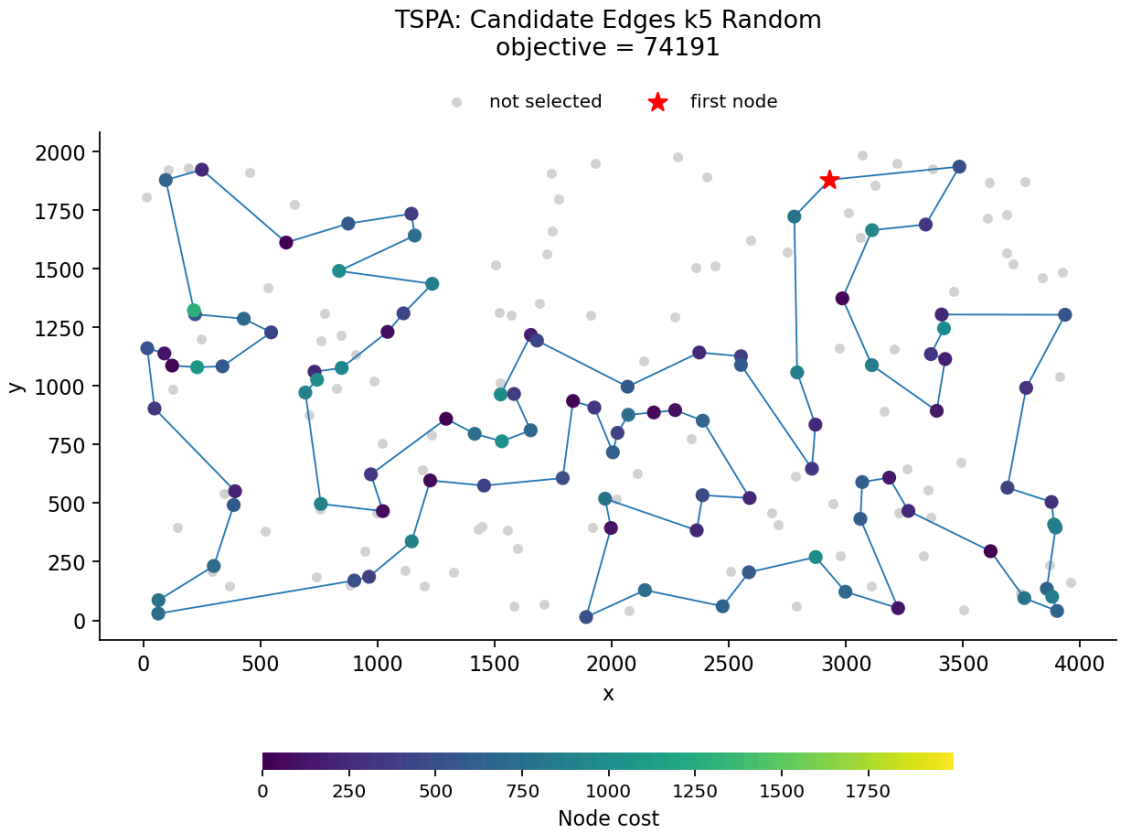


Solution (tour order):

```
  84 134 176 150 106  43 113  89  42 151
 138 182  57 107  88  78 193 153 170 159
 130  47  93  50   3 169   1  59 178 101
  29 136 186 164  85 174   8  54   9 143
  33 131  36 177  52 179  53 140 144  67
  16  51 139 112  71 157  91  80  48  31
  60 108 152  49  11  63 149 116 168  97
 127 161  30 160  83 123  90 132  76   4
 156  35 185 102  28 180 141 187 171   2
 189  87  45  73 135 114 181 192 158  58
```

Candidate Edges k5 Random

Objective: **74191**.

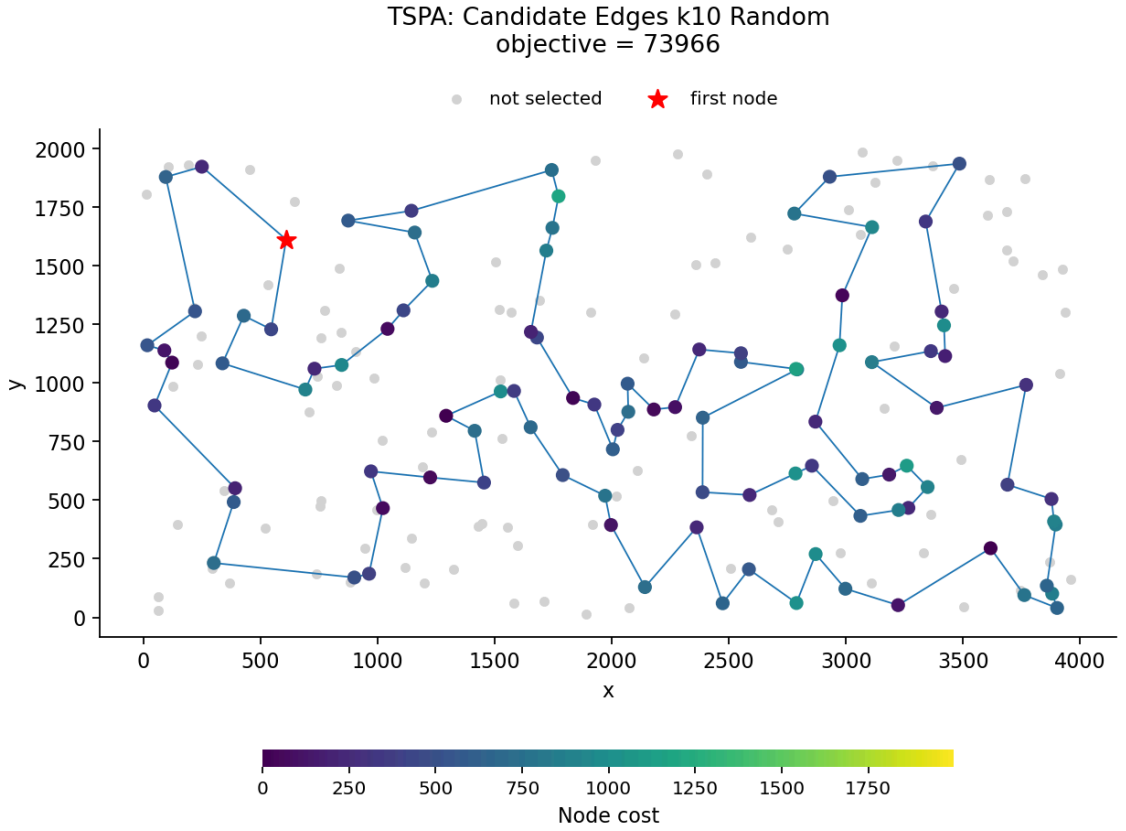


Solution (tour order):

```
136  29 178 184 164  85  93  50   3 169
  1 147  47 130 159 146 170 153 193  78
 88 107  89 106 150 151  57 182 138 158
192 181 114 135  73  58  84 134 131  36
177 179  53 140 144  67  45  87 141 189
  2 171 104 129  69 127  97 168 116 149
 63  11  49 152 108  60  82  31  48  80
 91 157  71  38   4  76 132  90 123  83
160 161 187  30 180  28 102 185  35 156
 51  16  52  33   9  54 176 174   8 186
```

Candidate Edges k10 Random

Objective: **73966**.

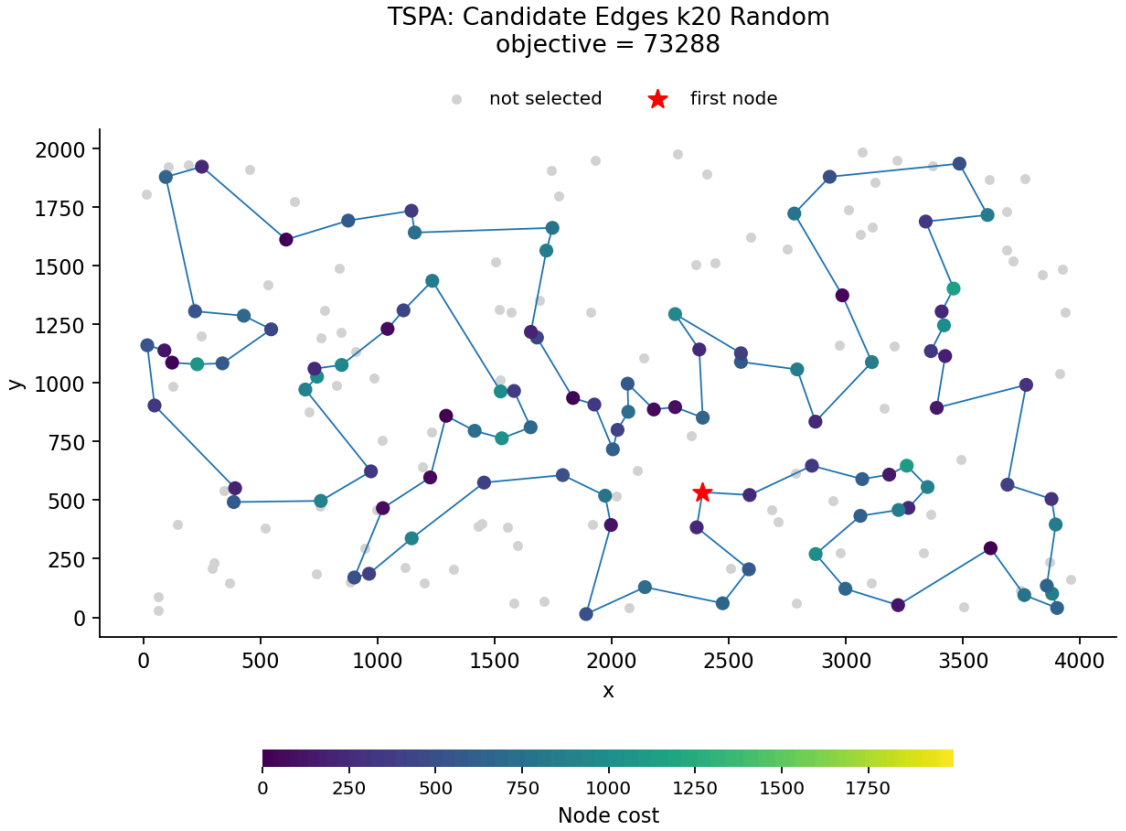


Solution (tour order):

```
 80  48  31  60 116 149  63 168  97 127
 69 171   2 187  30 141  87  28 180 156
 35 185  45  73 135 181  58 192 158 111
138 182  57 107  88  78 193 153 170 146
159 130  47  93  85   3  50 169   1 178
 29 136 186 184 164 142 174 150 106  43
113  89  42 151 176  72 134  84 131   8
194  54   9  33  36 177  52 179  53 140
144  67  16  51 139 112   5 115 157  91
 71   4  76 132  90 123 160  49 108 152
```

Candidate Edges k20 Random

Objective: **73288**.



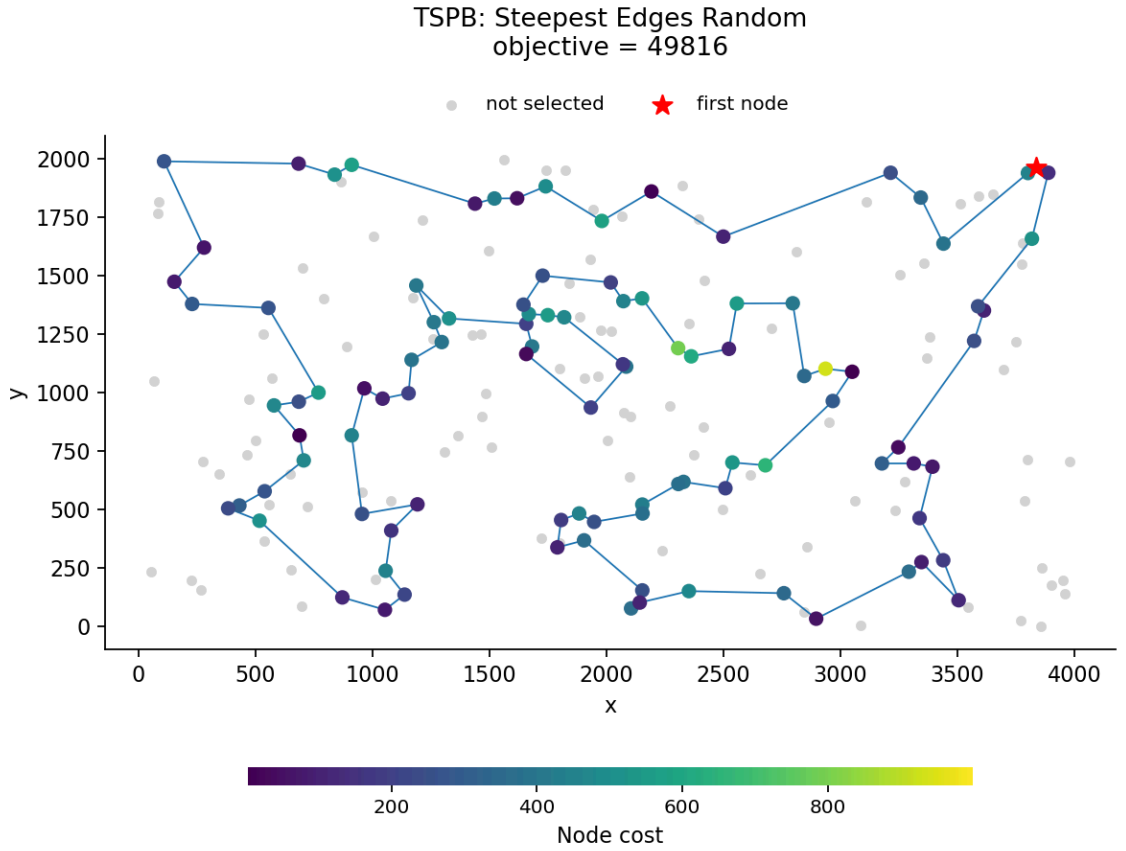
Solution (tour order):

```
  84 134 176 150 106  43 113  89  42 151
 138 182  57 107  88  78 193 153 170 159
 130  47  93  50   3 169   1  59 178 101
  29 136 186 164  85 174   8  54   9 143
  33 131  36 177  52 179  53 140 144  67
  16  51 139 112  71 157  91  80  48  31
  60 108 152  49  11  63 149 116 168  97
 127 161  30 160  83 123  90 132  76   4
 156  35 185 102  28 180 141 187 171   2
 189  87  45  73 135 114 181 192 158  58
```

TSPB

Steepest Edges Random

Objective: **49816**.

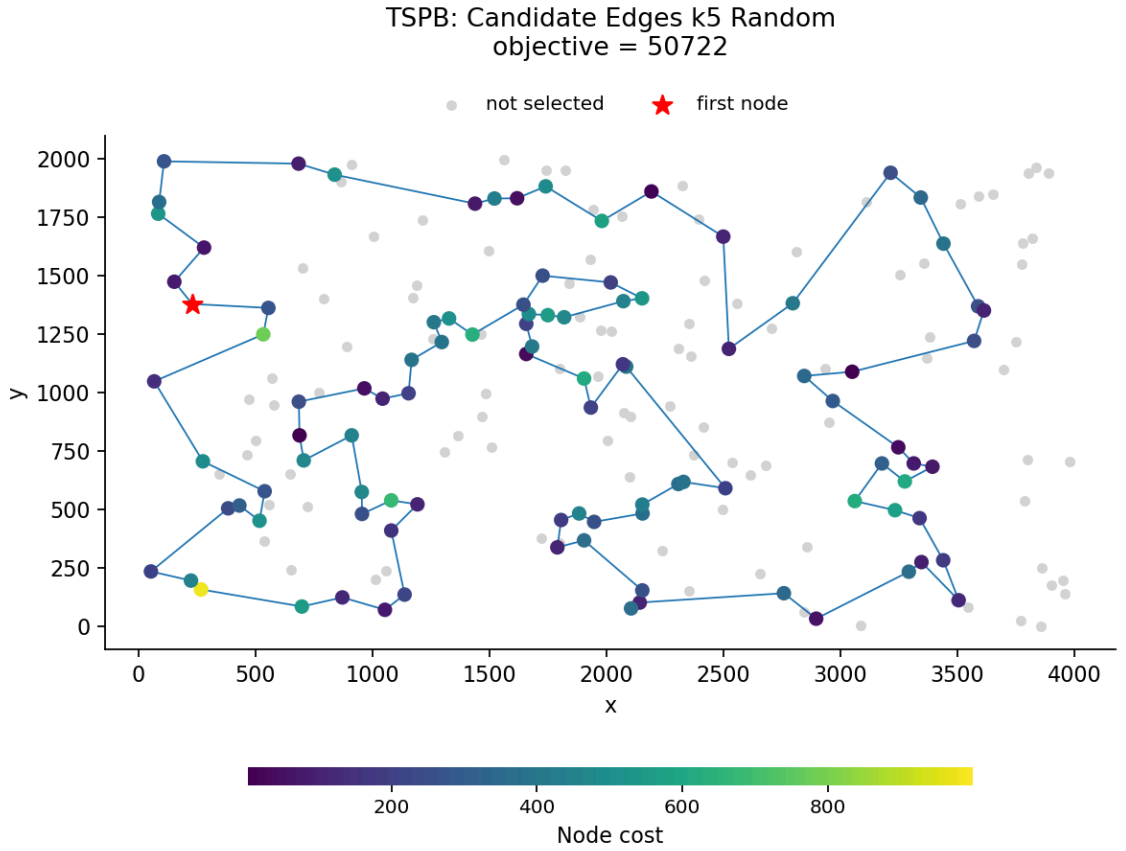


Solution (tour order):

```
 94  81 187  70 190  57  89 156  48  96
198 121  42 147 116  37 149  28 145 158
 26 125 177 170 115   6  29 132 197 175
134  45 157  60  40  71  97 154 140  47
 38  21  30 130 183 172 160  69 151   3
184 117   8 136 133 120  72  46  86  75
135  76 185 104 162 159  54 192  84  66
 67  24  34 152 181 167  73  59  85  51
179  82  87  88  99 171  74  20   2  14
153  90 129  80 127  10 114 196  61 105
```

Candidate Edges k5 Random

Objective: **50722**.

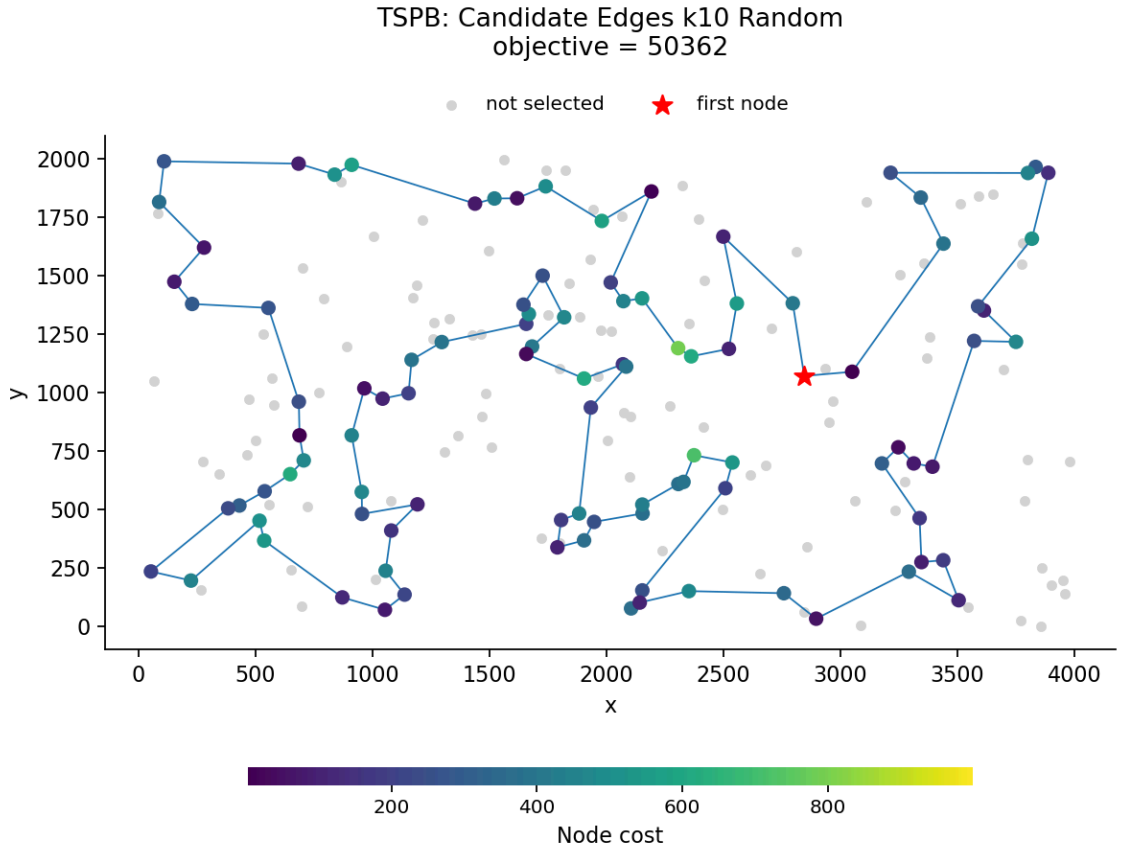


Solution (tour order):

```
145  28 149  13 146  37 116 147 121 198
 96  48 156  89  57  76 104 190  70 187
196 114  10  54 162 192 127 129  90  52
 80 150 182 153  14   2  20  74 171  99
 87  82 179  51  85  59  73 167 181 152
 34  24  67   3 184 151 122  69 160 172
136   8 117  46  86  72 120 133 189 183
 30  21  38  47 140 154 125 170 115  97
155  71 141  40  60  45 134 175  62 108
174   1 132  29 197   6 178  65 138 158
```

Candidate Edges k10 Random

Objective: **50362**.

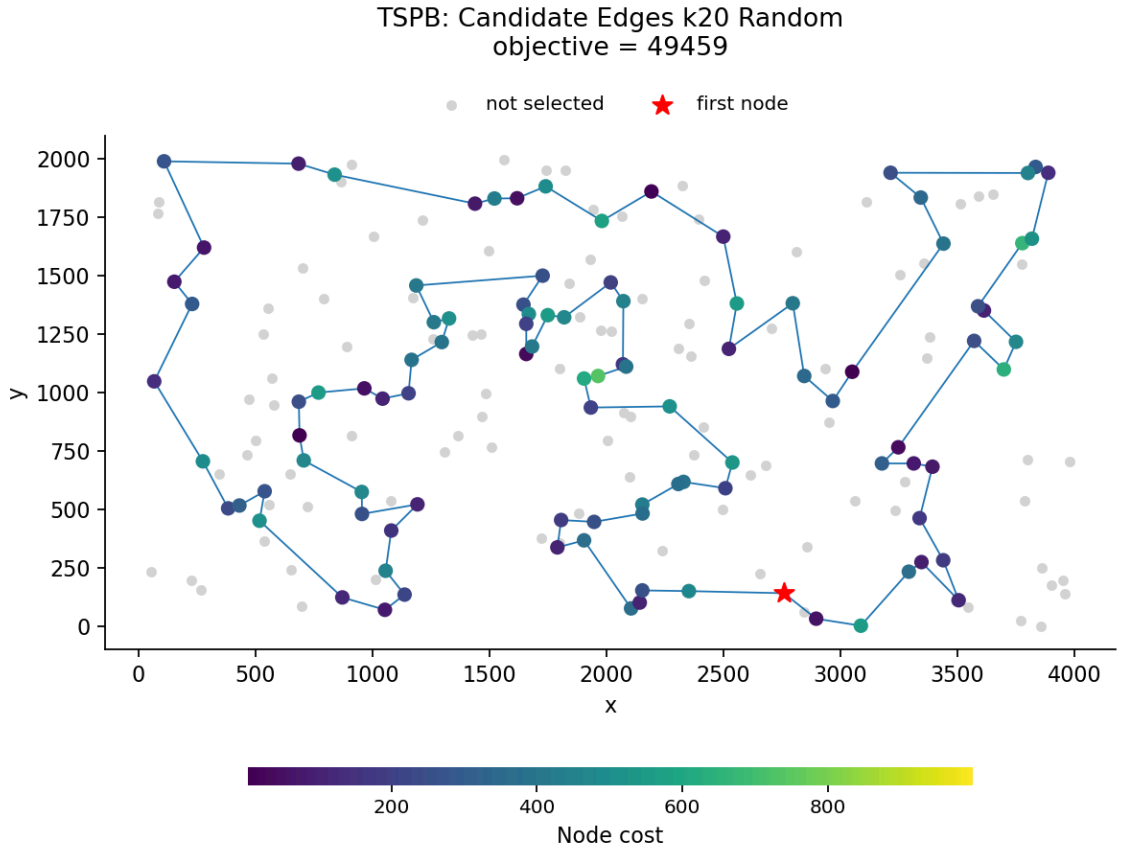


Solution (tour order):

```
162 104  57 185  76 135  75  86  46  72
 89 156  48  96 198 121  42 147 116  37
146 149  28 145 158 125 170 115 180   6
 29 132   1 174 197  36 175 134  45 157
 60  40  71 155  97 154 140  47  38  21
172 136 133 120 117 160  69 122 184   3
151  73  59  85  51 167 181 152  34  24
 11  66  67 179  82  87  88  99 171  74
  2  14  20 153  80 127 129  90  10 148
114 196  61 105  94  81 190  70 187  54
```

Candidate Edges k20 Random

Objective: **49459**.



Solution (tour order):

```
 99 171  17  74  20   2  14 153  90 129
 80 127  10 143 148 114 196 139  61 105
 94  81 190  70 187  54 192 162 104  76
185  57  89 156  48  96 198 121 147 116
 37 149  28 145  65 178 132  29   6 197
175 134  45 157  60  40  71 155 115 170
125  26 154 140  47  38  21 183  30 130
120 133 136 172  69 160   8 117  72  46
184   3 173 122 151  27  66  67  24  34
152 181 167  59  85  51  82  87 179  88
```

Solution checking

All 1600 exported solutions were independently checked in Python: exactly 100 distinct node indices in 0-199, and recomputed rounded-distance objectives equal the incremental C++ objectives. All CSV minima, maxima, and means were recomputed. All eight plotted best solutions match the summary minima.

Source code

Repository: [PiotrFoltyniewicz/Evolutionary-Computation](). The implementation and exported evidence are included in this project:

- Candidate lists, move generation, deltas, and steepest search
- Experiment runner
- Unchanged full-neighborhood baseline
- Validation and report generator
- All runs
- Summary
- Best solutions
- Lab 3 report
- Lab 3 summary
- Lab 2 summary
- Lab 1 summary

Conclusions

The required k = 10 gives a 4.27x speedup on TSPA and 5.10x on TSPB, with mean objective changes of +0.199% and -0.137%, respectively, relative to the lab 4 baseline.

k = 5 is fastest on both instances. The best candidate means come from k = 20 on TSPA (76095.42) and k = 10 on TSPB (52640.37), both slightly below the baseline but above the best means from earlier labs (72983.31 and 52513.55).

Candidate lists provide a useful time-quality tradeoff on these instances. Larger k does not guarantee better results because it changes the search path, and candidate-local optima may still admit improving non-candidate moves.